Laboratory Series No. 02

Dr. Rochdi Boudjehem

2025-10-25

Lab 1 — Amdahl & Gustafson: Measured Speedup and Efficiency

This lab builds directly on Lab 0's baseline. You will create a parameterized workload with a controllable parallel fraction p, measure speedup S(T) and efficiency E(T) across thread counts, and compare empirical results with Amdahl's and Gustafson's predictions.

```
References:
- Chapter 1 (performance metrics) and Chapter 2 (paradigms) slides. See "Environment & Set up" below (Windows cmd.exe with MinGW-w64/OpenMP emphasized).
- This lab focuses on OpenMP, which we will see in chapter 2. However, you can start familiarizing yourself with OpenMP pragmas and environment variables in practice.
```

---

1) Learning Objectives

- Implement a controlled mix of serial and parallel work to emulate different parallel fractions p.
- Sweep thread counts T ∈ {1,2,4,8,16} and p ∈ {0.3, 0.6, 0.9}; measure wall-clock time.
- Compute empirical Speedup S(T) = T1 / TT and Efficiency E(T) = S(T)/T.
- Compare measurements with:
  - Amdahl's Law (fixed problem size): S_A(T) = 1 / (S + P/T)
  - Gustafson's Law (scaled problem size): S_G(T) = T − S × (T − 1)
- Discuss deviations due to overheads, scheduling, memory hierarchy, and OS noise.

---

2) Environment & Setup (recap)

Choose one option and verify toolchains (from Lab 0):

- Windows (cmd + MinGW-w64): ensure `...\mingw64\bin` in PATH; check `gcc --version`.
- Windows WSL (Ubuntu): `sudo apt install build-essential` (OpenMP in GCC by default).
- Linux (native): GCC installed.

OpenMP compile/run examples (cmd.exe):

```
set CC=gcc
%CC% -O3 -march=native -fopenmp lab1_speedup.c -o lab1_speedup.exe
set OMP_NUM_THREADS=8
lab1_speedup.exe --p 0.6 --work 200000000
```

Optional: persist threads per run by passing `--threads N` rather than environment variable.

---

3) Task Description

1. Write a C/OpenMP program that:

- Accepts command-line parameters:
  - `--p <0..1>` parallel fraction (default 0.6)
  - `--threads <int>` thread count (optional; if omitted, use `OMP_NUM_THREADS` or 1)
  - `--work <int>` total iteration budget (default e.g., 200,000,000)
  - `--repeat <int>` number of timing repetitions (default 3)
- Splits the total work into:
  - Serial part = (1−p) × work, executed on a single thread
  - Parallel part = p × work, executed with `#pragma omp parallel for`
- Performs a simple floating-point operation inside the loop to avoid dead-code elimination (e.g., fused accumulate).
- Measures elapsed time with a high-resolution timer and prints: T (threads), p, total work, time (s), computed S(T), E(T) when T=1 baseline is known.

2. Measurement plan:
   - Fix total work so single-thread runtime is between ~0.5–2.0 s for stable runs.
   - For each p ∈ {0.3, 0.6, 0.9} and T ∈ {1,2,4,8,16}:
     - Run program (repeat 3×), record best and average time.
     - Compute S(T) and E(T).
3. Theoretical comparison:
   - Using measured serial fraction S=1−p, compute:
     - Amdahl prediction $S_A(T) = 1 / (S + (1−S)/T)$
     - Gustafson prediction $S_G(T) = T − S × (T − 1)$
   - Put measured vs predicted values in a table.
4. Analysis prompts:
   - When does measured S(T) fall below Amdahl? Why? (overheads, memory, load imbalance)
   - Why can Gustafson's be more optimistic? (scaled problem assumption)
   - Identify a configuration with highest efficiency and explain why.

---

4) Starter Code (C/OpenMP)

Save as `lab1_speedup.c`:

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <math.h>
#include <omp.h>
#ifdef _WIN32
#include <windows.h>
static double now_s(){ LARGE_INTEGER f,c; QueryPerformanceFrequency(&f); QueryPerf
ormanceCounter(&c); return (double)c.QuadPart/(double)f.QuadPart; }
#else
#include <time.h>
static double now_s(){ struct timespec ts; clock_gettime(CLOCK_MONOTONIC, &ts); re
turn ts.tv_sec + ts.tv_nsec*1e-9; }
#endif

static void busy(float *acc, uint64_t iters){
```

```
    // Simple compute to avoid being optimized out
    float a = *acc;
    for(uint64_t i=0;i<iters;i++){
        a = a * 1.0000001f + 1.0f; // cheap FLOPs
        if ((i & 0xFFFFF) == 0) a = fmodf(a, 1000.0f);
    }
    *acc = a;
}

int main(int argc, char** argv){
    double p = 0.6;              // parallel fraction
    int threads = 0;             // 0 => use OMP env or default
    uint64_t work = 200000000ULL; // total iteration budget
    int repeat = 3;

    for(int i=1;i<argc;i++){
        if(!strcmp(argv[i],"--p") && i+1<argc) p = atof(argv[++i]);
        else if(!strcmp(argv[i],"--threads") && i+1<argc) threads = atoi(argv[++i]
);
        else if(!strcmp(argv[i],"--work") && i+1<argc) work = (uint64_t) atoll(arg
v[++i]);
        else if(!strcmp(argv[i],"--repeat") && i+1<argc) repeat = atoi(argv[++i]);
        else if(!strcmp(argv[i],"--help")){
            printf("Usage: %s [--p 0..1] [--threads N] [--work W] [--repeat R]\n",
argv[0]);
            return 0;
        }
    }
    if(p < 0.0) p = 0.0; if(p > 1.0) p = 1.0;

    if(threads > 0) omp_set_num_threads(threads);
    int T = omp_get_max_threads();

    uint64_t par_iters = (uint64_t)(p * (double)work);
    uint64_t ser_iters = work - par_iters;

    double best = 1e100, sum = 0.0;
    float acc = 0.0f;

    for(int r=0;r<repeat;r++){
        float tmp = acc;
        double t0 = now_s();
        // Serial section
        busy(&tmp, ser_iters);
        // Parallel section
        #pragma omp parallel for schedule(static)
        for(int64_t i=0; i<(int64_t)par_iters; i++){
            float loc = (float)i + tmp;
            busy(&loc, 1); // one inner iteration per outer, adjust with --work
        }
        double t1 = now_s();
        double dt = t1 - t0;
        if(dt < best) best = dt;
        sum += dt;
    }
    double avg = sum / repeat;
```

```
    printf("p=%.2f, threads=%d, work=%llu, time_best=%.6f s, time_avg=%.6f s\n",
           p, T, (unsigned long long)work, best, avg);

    // Note: S(T), E(T) require T1 reference (from T=1 run). Compute in analysis.
    return 0;
}
```

Build (cmd.exe):

```
set CC=gcc
%CC% -O3 -march=native -fopenmp lab1_speedup.c -o lab1_speedup.exe
```

Run examples (cmd.exe):

```
set OMP_NUM_THREADS=1
lab1_speedup.exe --p 0.6 --work 200000000 --repeat 3
set OMP_NUM_THREADS=8
lab1_speedup.exe --p 0.6 --work 200000000 --repeat 3
lab1_speedup.exe --p 0.3 --threads 16 --work 300000000
```

Tips: - Tune `--work` so that the 1-thread run is ~1 s on your machine. - Use `--repeat` to stabilize timing; report both best and average.

---

5) Data Collection Templates

Recommended CSV (copy into `results.csv`):

```
p,T,best_s,avg_s,S_meas,E_meas,S_amdahl,S_gustafson
0.30,1,1.230,1.250,1.00,1.00,1.00,1.00
0.30,2,0.800,0.820,1.54,0.77,1/(0.7+0.3/2), 2-0.7*(2-1)
...
```

Markdown table template:

p	T	time_best (s)	S_meas	E_meas	S_A (Amdahl)	S_G (Gustafson)
0.30	1		1.00	1.00	1.00	1.00
0.30	2					
0.60	8					
0.90	16					

Computations: - With S = 1 − p and P = p: - Amdahl: S_A(T) = 1 / (S + P/T) - Gustafson: S_G(T) = T − S × (T − 1) - Measured: S_meas = T1 / TT, E_meas = S_meas / T

---

6) Troubleshooting

- If timings are noisy, increase `--work` or `--repeat`.
- If speedup is <1 at high T, check for oversubscription (too many threads), thermal throttling, or memory bandwidth limits.
- On Windows cmd.exe, set threads using `set OMP_NUM_THREADS=N` or `--threads N`.
- If GCC can't find OpenMP, ensure you're using MinGW-w64 build of GCC and not an ancient toolchain.

- Avoid `sleep` in the serial section; use compute loops for portability and consistent timing.

Happy scaling — this lab will ground your understanding of speedup laws with real measurements.